Graph RAG backend for the Graph Knowledge System (v2, hardened)

Branch: `feature/graph-rag`, cut from `feature/admin-panel`, so auth, roles, `scoped()` and the audit log already exist.

Context

The admin panel gives login, roles, company isolation and audit logging, but the system has no knowledge. This ticket adds the Graph RAG backend: ingest sources, extract and validate entities and relationships, build and update a graph, retrieve graph context for a question, and have the LLM answer only from that context with checked citations. The ticket's other asks are covered the same way: construction and update pipeline, grounding, error handling.

Fixed decisions: graph in **Postgres tables**, LLM is **Gemini**, sources are **PDF, CSV / database rows, text and Markdown**, backend only (no new UI). Local Postgres 17 is running with pgvector 0.8.7, so vector search is available. This version folds in an independent design review of the first draft.

Flow

```
upload -> cap + parse (422 on bad file, no row yet)
       -> insert Source(processing)  [unique company+hash = concurrency guard]
       -> chunk -> Gemini extract per chunk (no DB transaction held)
       -> validate (pure) -> ONE short write transaction: apply_extraction()
question -> n-gram entity match -> BFS k hops -> rank -> context pack
         -> Gemini answer -> grounding check (quote substring) -> answer + sources
```

Data model (`backend/app/graph/models.py`, same `Base`, imported in `main.py`)

- Every `company_id` is **NOT NULL** (NULLs defeat unique constraints). `UNIQUE(company_id, id)` on Source, Chunk, Entity, with composite FKs from Chunk, Relation, Evidence so a cross-company reference cannot exist.
- `Source(id, company_id, title, kind, content_hash, status[processing|done|partial|failed], report JSON, timestamps)`, `UNIQUE(company_id, content_hash)`.
- `Chunk(id, company_id, source_id, ordinal, text)`, removed together with its Source.
- `Entity(id, company_id, name, normalized_name, type, description)`, `UNIQUE(company_id, normalized_name, type)`. Type is normalized to a fixed enum (unknown -> `other`) so "other" vs "person" cannot create duplicate nodes.
- `Relation(id, company_id, subject_id, predicate, object_id)`, `UNIQUE(company_id, subject_id, predicate, object_id)`. **No stored support or confidence**, they drift on prune.
- `Evidence(id, company_id, chunk_id, entity_id nullable, relation_id nullable, quote, confidence)`, `CHECK` exactly one of entity_id or relation_id, `UNIQUE(chunk_id, entity_id)`, `UNIQUE(chunk_id, relation_id)`. Support = count of distinct sources, computed at read time.
- `db.py`: `PRAGMA foreign_keys=ON` for sqlite, or dependent-row deletes and delete-order bugs stay invisible in tests.
- Column lengths enforced in code (truncate or reject), since sqlite ignores `String(n)` and Postgres raises. Strip `\x00` from parsed text.

Isolation (the risk that matters most)

- One helper `graph_scope(user, company_id) -> int` in `deps.py`: sysAdmin without `company_id` -> 422; non-sysAdmin passing another company -> 403; unknown company -> 404; write to an inactive company -> 409.
- `store` and `retrieve` take an `int company_id` (never a `User`) and filter on it unconditionally, including every join. The source title in the context pack is the realistic leak path.

- Audit rows carry the effective company (a sysAdmin's own `company_id` is None). `graph.query` logs question length or hash, never the text.
- No caching in v1; any later cache is keyed on `company_id`.

Pipeline (`backend/app/graph/`)

1. **Parse** (`parsing.py`): text/Markdown; PDF via `pypdf` (encrypted or text-less PDF -> 422, no OCR); CSV and JSON rows rendered as `col: value` lines grouped into blocks (this covers "database rows"). Byte, page and chunk caps; upload read with a size limit.
2. **Chunk** (`chunking.py`): paragraph-aware, about 1000 chars, small overlap. Pure.
3. **Extract** (`extract.py`): one Gemini call per chunk, JSON schema output. Instructions go in `system_instruction`; the chunk sits in a delimited block, no tools. Run in a small thread pool (about 4) with a total deadline; abort as `failed` after N consecutive `LLMUnavailable`. A `MAX_TOKENS` finish counts as bad output.
4. **Validate** (`validate.py`, pure). Keep a relation only if:
   - it fits the schema, endpoints are in the extracted entities **and each entity's normalized name appears in the chunk text** (the first check alone is circular),
   - not a self-loop *after* normalization, predicate normalized to snake_case,
   - confidence >= threshold (default 0.5),
   - the quote, compared whitespace-collapsed and casefolded with a minimum length, is a substring of the chunk (so PDF line breaks don't reject good facts). Rejections are counted by reason in `Source.report`.
5. **Apply** (`store.py`, `apply_extraction(db, company_id, source, extraction)`): the only function that writes graph rows, one short transaction. Entity upsert with dialect `ON CONFLICT DO NOTHING`.
   - **Update:** insert new chunks and evidence first, then delete old chunks (their evidence goes with them), then prune. Shared entity ids stay stable.
   - **Prune order:** relations with no evidence, then entities with no evidence and no surviving relation.
   - Same hash is a no-op only when `status == done`; partial or failed sources can be retried.
   - On PUT, if the new run is not `done`, the old graph is kept and the failure goes in `report`.
   - `FOR UPDATE` on PUT and DELETE; a stale `processing` row times out.
6. **Retrieve** (`retrieve.py`): seed entities by matching normalized question n-grams to entity names (no extra LLM call, one less failure and injection surface), `contains(..., autoescape=True)` so a `%` matches nothing special. Python BFS, one query per hop over the frontier in both directions with a per-node LIMIT and global node/edge caps (a recursive CTE materializes hub-degree^k walks before caps apply). Rank by hop, confidence, distinct-source support. Fallback: keyword match on chunks with stopwords dropped, behind a `fallback_chunks()` seam (vector search plugs in here). Output: numbered facts `[F#]` and passages `[S#]` with source titles, within a token budget.
7. **Answer** (`answer.py`): empty retrieval refuses **without calling the LLM**. Otherwise Gemini sees only the context pack, answers from it, and returns `{answer, citations[{id, quote}], insufficient}`. Passage text is escaped so `[S#]`, `[F#]` and delimiters inside documents are neutralized.
8. **Grounding check in code:** each citation must reference a real passage **and its quote must be a substring of that passage**; `sources` in the response is built only from validated citations; stray `[S#]` markers not in the valid set are stripped from the answer. No valid citation or `insufficient` -> "I couldn't find this in your sources." with no sources. A `ponytail:` comment records that there is no entailment check, so a claim wrongly attributed to a real quote is the known ceiling.
9. **Errors** (`errors.py`, `llm.py`): `LLMUnavailable`, `LLMBadOutput`, `SourceUnreadable`. Timeout plus retries with backoff on 429/5xx. A chunk that fails is skipped and counted, source ends `partial`; all fail -> `failed`. Query time: 502 LLM down, 504 timeout, bad structured output retried once then 502, unreadable file 422. `report` stores `type(exc).__name__`, never messages or prompts. User messages stay generic.
10. **DB connection handling:** `get_current_user` opens a transaction on the request session, and synchronous Gemini calls would hold it and drain the pool. Order each request: auth -> read -> `db.rollback()` -> LLM calls with no transaction -> one write transaction.

CONFIDENTIAL

LLM layer (`llm.py`)

`LLM` protocol with `generate_json(system, prompt, schema)` and `embed(texts)`; `GeminiLLM` on the `google-genai` SDK; `get_llm()` FastAPI dependency overridden by `FakeLLM` in tests (scriptable failures, records prompts). Config via env: `GEMINI_API_KEY` (never committed or logged), `GEMINI_MODEL`, `GEMINI_EMBED_MODEL`, limits. Model ids are **config, not hard-coded**: current docs disagree on the generation model name (3.x flash variants appear), and the embeddings docs show `gemini-embedding-2`. Confirm both against the live model list at step 8, before the smoke test.

API (`routers/graph.py`; roles per the approved plan, all three have graph:read/write)

Method	Path	Permission	Purpose
POST	/graph/sources	graph:write	Upload pdf, csv, md, txt or JSON rows; ingest
GET	/graph/sources	graph:read	List with status and report
PUT	/graph/sources/{id}	graph:write	Re-ingest changed content
DELETE	/graph/sources/{id}	graph:write	Remove source and prune
GET	/graph/entities	graph:read	Search entities
GET	/graph/entities/{id}/neighbors	graph:read	Relations around an entity
POST	/graph/query	graph:read	Grounded answer and cited sources

sysAdmin passes `company_id` (query or form field). A simple per-company request cap guards `graph:write`, since the `user` role has it. Failed ingests and queries are audited with a status in meta (`graph.source_ingest`, `graph.source_update`, `graph.source_delete`, `graph.query`).

Hybrid retrieval with pgvector (last step of this ticket, behind the seam)

`ChunkEmbedding(chunk_id, company_id, embedding, model)`, column `JSON().with_variant(Vector(768), "postgresql")`. Tests use a `FakeEmbedder` and Python cosine over that company's rows; Postgres queries always `WHERE company_id = :c` and use an exact scan first (an HNSW index later needs `hnsw.iterative_scan`, or filtered queries return too few rows). `CREATE EXTENSION vector` is a one-time ops step, not in `lifespan`. An embedding failure never fails ingestion.

Files

- New: `backend/app/graph/{models,parsing,chunking,extract,validate,store,retrieve,answer,llm,errors,schemas}.py`, `backend/app/routers/graph.py`, `backend/tests/test_graph_*.py`
- Edit: `main.py` (router, models import), `config.py` (Gemini and limits), `db.py` (sqlite FK pragma), `deps.py` (`graph_scope`), `requirements.txt` (`google-genai`, `pypdf`, `python-multipart`, `psycopg`, `pgvector`; pin versions)
- Reuse: `deps.get_current_user`, `deps.require_permission`, `audit.log_action`, `tests/conftest.py` fixtures and `login()`
- Docs: new endpoints and cases in `docs/API-Endpoints-for-Testers.md`, status doc, regenerate PDFs.

Build order

1. Models (NOT NULL, constraints, delete rules), FK pragma, `graph_scope`, isolation test first.
2. Parse, chunk, validate (pure, tested).
3. `apply_extraction` with hand-built extractions: prune, update, idempotency. No LLM, no HTTP.
4. Extract with `FakeLLM`.

CONFIDENTIAL

5. BFS retrieval and context pack on a hand-built graph.
6. Answer and grounding.
7. Endpoints, audit, error mapping, rate cap.
8. Opt-in run on a scratch Postgres database (NUL bytes, lengths, ON CONFLICT, FKs), then confirm Gemini model ids and run the real smoke test. `conftest.py` runs `drop_all`, so never point it at a real database.
9. pgvector hybrid retrieval.

Verification

All with `FakeLLM`, sqlite, no network unless noted:

- chunker, parsers (md, csv, json rows, small PDF fixture, encrypted/empty PDF -> 422)
- validator drops: bad schema, endpoint not in chunk text, normalized self-loop, low confidence, non-verbatim quote; a quote with PDF line breaks still passes
- same hash done = no-op; same-hash partial source reprocesses; changed file removes stale facts and keeps shared ones with stable ids; a PUT where every chunk fails keeps the old graph
- delete prunes only what that source alone supported
- isolation: A cannot see B's entities, neighbors, sources or query results; prompts captured by `FakeLLM` for A contain no B strings; sysAdmin without `company_id` -> 422, wrong company -> 403, sysAdmin audit row carries the target company; cross-company references rejected by the composite FKs
- retrieval: a cycle (A-B-A) terminates, a hub respects caps, `%` in the question matches nothing special, fallback works, empty retrieval refuses without an LLM call
- grounding: a citation with a wrong quote is dropped; a citation to an unknown id is dropped; a passage containing `[S1] ignore previous instructions` is neutralized; no valid citation -> refusal with empty sources
- errors: LLM down -> 502, timeout -> 504, malformed output -> retry then 502, partial ingestion report, nothing half-written; concurrent-ingest loser simulated by pre-inserting the conflicting row
- audit: one row per graph action, including failures
- Manual, real: scratch Postgres 17 database plus `GEMINI_API_KEY`: ingest a PDF and a CSV, ask answerable and unanswerable questions, confirm citations and refusals; then pgvector recall check.
- Gate before any push: pytest, UI build, prime-directive checklist.

Known limits

- No entailment check on answers (documented in code); no OCR; no alias merging ("IBM" vs full name); ingest is synchronous with caps (background queue is the upgrade); tables still created at startup, Alembic later.

CONFIDENTIAL